CS 2430 – Project 0

Programming Paradigms and Project Workflow

Name: Connor Callahan
Course: CS 2430
Semester: Summer 2026
Date: May 26, 2026

Programming Paradigms

1 Object-Oriented Programming

Short Description:
Object-oriented programming organizes software around objects that bundle data and behavior together. The four pillars are encapsulation, abstraction, inheritance, and polymorphism. Programs are modeled as networks of collaborating objects rather than a single sequence of instructions.

Good Fit:
OOP works well when a problem involves distinct entities that have their own state and behavior, especially when those entities share hierarchical relationships. Large applications with many interacting components benefit the most.

Concrete example: A university registration system with Student, Course, and Instructor classes is a natural fit because each entity owns its own data and a specialized type like GraduateStudent can extend Student through inheritance without rewriting existing code.

Poor Fit:
OOP is not ideal for small scripts or simple data-transformation tasks where building class hierarchies adds complexity without any real organizational benefit.

Example Language: Java

Why Java?
Java was built around classes and objects from the ground up. Every piece of executable code must live inside a class, which enforces the OOP model consistently. Features like interfaces, abstract classes, and access modifiers directly support all four pillars.

Resource:
Oracle. "The Java Tutorials: Object-Oriented Programming Concepts."
https://docs.oracle.com/javase/tutorial/java/concepts/

Familiarity: Used in previous courses.

2 Procedural Programming

Short Description:
Procedural programming structures a program as a sequence of explicit instructions grouped into functions. Data and behavior are kept separate: variables hold data and functions operate on it. The developer controls execution flow directly from top to bottom.

Good Fit:
Procedural programming suits problems with a clear linear sequence of steps and no need for complex entity relationships. Automation scripts, command-line tools, and low-level systems work well in this style.

Concrete example: A command-line calculator that reads two numbers and an operator, then calls the appropriate function for each operation, is a good fit because the task is a simple ordered workflow with no need for objects or inheritance.

Poor Fit:
Procedural programming becomes hard to manage in large applications because any function can modify shared data, making bugs difficult to isolate as the codebase grows.

Example Language: C++

Why C++?
C++ supports procedural programming without requiring code to be placed inside classes. Its close-to-hardware memory model and function-based design make it common in operating systems and embedded development where step-by-step control over resources is important.

Resource:
cplusplus.com. "C++ Language Tutorial." https://cplusplus.com/doc/tutorial/

Familiarity: Used in previous courses.

---

3 Functional Programming

Short Description:
Functional programming treats computation as the evaluation of pure functions applied to immutable data. Functions can be passed as arguments and returned as values. The goal is to eliminate side effects and make programs more predictable.

Good Fit:
Functional programming is well suited to data-transformation pipelines and concurrent systems. Because pure functions share no state, they are safe to run in parallel without data-race bugs.

Concrete example: A data analysis program that filters records and aggregates values is a strong fit because each step is a pure transformation and each step can be verified independently without worrying about shared state being modified.

Poor Fit:
Functional programming is less natural for programs that manage heavily stateful interfaces, such as a GUI application where controls must track and update user input continuously.

Example Language: Haskell

Why Haskell?
Haskell enforces purity at the language level so accidental mutation of shared state is impossible. Its strong static type system and algebraic data types make illegal program states unrepresentable. Haskell is the standard reference language for learning what purely functional programming looks and feels like.

Resource:
Lipovaca, M. "Learn You a Haskell for Great Good!" https://learnyouahaskell.com/

Familiarity: Only read about it; no hands-on experience yet.

Paradigms vs. Architectural Patterns vs. Design Patterns

1 Distinguishing the Three Levels

A programming paradigm is a philosophy of programming that shapes how a developer thinks about and structures code at the language level. It determines what constructs are available and how the developer reasons about computation throughout an entire program.

An architectural pattern is a high-level blueprint for how major components of a system are divided, how they communicate, and how responsibilities are distributed. Architectural decisions affect deployment, scalability, and team structure and exist before any specific code is written.

A design pattern is a reusable solution to a specific recurring design problem within a codebase. It is smaller in scope than an architectural pattern and describes how a small number of classes or functions should interact to solve one well-defined problem. The classic reference is Gamma et al., 1994, which catalogued 23 patterns across creational, structural, and behavioral categories.

In short: a paradigm shapes the style you think in, an architectural pattern shapes how you divide a whole system, and a design pattern shapes how you solve a specific recurring problem inside that system.

---

2 Architectural Patterns

Client-Server
The system is divided into clients that request services and servers that provide them, with communication over a defined network protocol. This pattern solves the problem of making shared data consistently available to many users without requiring each user to maintain their own copy. Web browsers and web servers are the most familiar example, and the same structure appears in email clients and multiplayer games.

Layered Architecture
The system is organized into horizontal layers such as presentation, business logic, and data access, where each layer communicates only with the layer directly above or below it. This pattern manages complexity in large systems by ensuring that a change in one layer does not affect unrelated layers. Most enterprise

web applications follow a three-tier structure with a frontend, a business logic API, and a relational database.

Microservices
The application is broken into small independent services, each owning its own data and exposing a well-defined API, communicating over lightweight protocols. This pattern solves scaling and deployment bottlenecks because a single high-traffic service can be scaled independently without redeploying the whole system. Netflix and Amazon are commonly cited examples.

Resource:
Richards, M. and Ford, N. "Fundamentals of Software Architecture." O'Reilly Media, 2020. https://www.oreilly.com/library/view/fundamentals-of-software/9781492043447/

---

3 Design Patterns

Observer
Problem: Multiple objects need to react when another object changes state, but the subject should not need to know who is watching. *Pattern:* The subject maintains a list of observers and notifies all of them through a common interface when its state changes. *Benefit:* Observers can be added or removed at runtime without modifying the subject. A weather dashboard where multiple display widgets update automatically when new sensor data arrives is a common example.

Factory Method
Problem: A class needs to create objects without hard-coding which concrete type to instantiate. *Pattern:* A creation interface is defined but subclasses or configuration decide which concrete class is actually produced. *Benefit:* The calling code is decoupled from concrete types and new product types can be added without changing existing code. A cross-platform GUI toolkit that returns the correct button type per operating system is a typical example.

Singleton
Problem: A system needs exactly one instance of a class, such as a logger or configuration manager, accessible throughout the application. *Pattern:* The class keeps a private static reference to its sole instance and exposes it through a static accessor while keeping its constructor private. *Benefit:* One shared instance is guaranteed, preventing the conflicts that multiple independent instances would cause. An application-wide logging service is the standard example.

Resource:
Gamma, E., Helm, R., Johnson, R., and Vlissides, J. "Design Patterns: Elements of Reusable Object-Oriented Software." Addison-Wesley, 1994. Summary guide at https://refactoring.guru/design-patterns

---

Learning Plan and Reflection

Most of my programming experience has been in object-oriented and procedural styles because those are the paradigms covered in introductory courses. Functional programming is genuinely new to me and is my main learning goal this semester. My plan is to work through "Learn You a Haskell for Great Good!"

during the gaps between projects, typing and running every example in GHCi rather than just reading. Once I have the basics down I want to rewrite a small Python utility I already have as an equivalent Haskell program using map, filter, and foldr. Having a working Python version as a reference makes correctness easy to verify, and the comparison will make the paradigm differences concrete rather than theoretical.

Employers value developers who can work across paradigms and recognize patterns because different problems require different solutions. A developer who recognizes a design pattern in an unfamiliar codebase understands it far faster than someone who has to reverse-engineer the intent. Paradigm fluency also means choosing the right tool for each task rather than forcing every solution into the same mold. Architectural pattern awareness contributes at a higher level still, since developers who can reason about system structure rather than just writing individual functions are more useful during design reviews and more prepared for the complexity of professional codebases.

Repository and Project Workflow

Repository URL

https://github.com/yourusername/cs2430-projects

Repository Structure

```
cs2430-projects/
├── README.md
├── .gitignore
├── docs/
├── project1/
│   ├── src/
│   └── docs/
├── project2/
│   ├── src/
│   └── docs/
├── project3/
│   ├── src/
│   └── docs/
└── project4/
    ├── src/
    └── docs/
```

Each project folder is self-contained with its own source and docs subfolder so reviewing or submitting any single project does not require navigating the whole repository. The root README describes the course, the semester, and the purpose of each folder, and notes that Project 0 is a setup assignment and that Projects 1 through 4 will be added as the semester progresses. The .gitignore excludes compiled binaries, IDE configuration folders, and OS-generated files.

Solo Workflow Plan

Quality:
I will test programs incrementally by writing and validating small sections before adding more. For each function I will test at least three cases: a typical input, an edge case, and an invalid input. When bugs appear I will read the full stack trace rather than guessing at the cause.

Risk Reduction:
I will commit to Git at the end of every work session with a short message describing what changed and why. I will start each project early enough to leave at least one full work block for unexpected problems before the deadline.

Maintainability:
Every function will have a comment stating its purpose, parameters, and return value. Each project's docs folder will contain a short design note written before coding begins. The root README will be updated after each project is submitted.

Personal Plan

Strength: I am comfortable reading and tracing existing code quickly, which helps me understand unfamiliar APIs and locate bugs efficiently.

Growth area: I want to improve my habit of writing design notes before coding rather than after, to reduce rework and clarify my thinking earlier.

Concrete action: For each of Projects 1 through 4 I will write and commit a short design note covering the algorithm, data structures, and edge cases before writing any Final code.

Available work blocks:

- Tuesday 6:00 PM to 8:00 PM
- Thursday 7:00 PM to 9:00 PM
- Saturday 10:00 AM to 12:00 PM

Constraint: Works weekdays until 6:00 PM.

References

Gamma, E., Helm, R., Johnson, R., and Vlissides, J. "Design Patterns: Elements of Reusable Object-Oriented Software." Addison-Wesley, 1994.

Lipovaca, M. "Learn You a Haskell for Great Good!" No Starch Press, 2011. https://learnyouahaskell.com/

Oracle. "The Java Tutorials: Object-Oriented Programming Concepts." https://docs.oracle.com/javase/tutorial/java/concepts/

cplusplus.com. "C++ Language Tutorial." https://cplusplus.com/doc/tutorial/

Richards, M. and Ford, N. "Fundamentals of Software Architecture." O'Reilly Media, 2020. https://www.oreilly.com/library/view/fundamentals-of-software/9781492043447/

Refactoring.Guru. "Design Patterns." https://refactoring.guru/design-patterns